PROVEIT RESEARCH NOTE

Further Lean Proof Simplifications

Four mechanisms beyond the existing DiophantineConsistency proposal corpus

Prepared for Vladimir Reshetnikov

26 October 2026

Pinned source

`9f35798db4a63c25e435fca14d75931144657d7e`

Proof environment

Lean 4.32.0; Mathlib revision

`81a5d257c8e410db227a6665ed08f64fea08e997`

Result. The most promising additional reduction is to prove the entire controller-unit family from one fixed modular certificate. A second change makes the full lifting-the-exponent divisibility spectrum the foundation of `PrimePowerOrder`; its complete candidate module is checked and removes five lines after charging the new helper. A four-line Pell proof removes four more lines. A fourth proposal characterizes an existing normalization by finite raw-cell prefixes separated by the established reset fields.
The arithmetic kernels compile. The LTE candidate and its existing focused checks compile. The Gamma repository adapter and the cyclic-cell adapter are clearly separated as integration work. No full repository rebuild, speedup, or new consistency-equivalence theorem is claimed.

Executive assessment

The requested baseline is unusually demanding: every idea in the existing proposal archive must be treated as already available, including optional and uncompiled proposals. This report therefore does not recommend another generic congruence API, carry record, decoder invariant, finite residual abstraction, coefficient certificate, or generated circuit proof.

The retained recommendations are concrete changes to the mathematical route of particular proofs. Their novelty is relative to the inspected repository corpus, not to number theory or the general Lean literature.

	Recommendation	Evidence and honest benefit
A	Evaluate the actual controller coefficient in one fixed quotient, then lift coprimality to every power modulus.	Complete arithmetic kernel checked; repository adapter uncompiled. Removes distinct order/Frobenius and factorwise obligations from the main argument.
B	Prove the arbitrary-exponent LTE divisibility spectrum first; derive exact lifting and order from it.	Full candidate module and focused checks pass. Whole module: 80 to 75 lines; six old theorem types preserved.
C	Characterize existing carries by finite raw-cell prefixes after a proved reset.	Five arithmetic lemmas checked. Cyclic-cell adapter proposed; no immediate line saving claimed.
D	Replace the induction in `sixteen_pow_bound` by a quadratic inequality in 4^r.	Exact theorem body checked; 8 to 4 proof lines, without a helper.

Recommended implementation order. Apply B and D as small proof-only changes, then integrate A while retaining public intermediate lemmas until their clients have been reviewed. Implement C first as a characterization theorem for existing carries. Replacing the already short fixed-point existence proof is not the recommended first step.

The measured production-source reduction is **nine physical lines across B and D**. This intentionally modest total includes the LTE helper and does not count hypothetical future savings, removed test files, or unintegrated Gamma wrappers. A's main attraction is a simpler dependency structure; C's is local reasoning about changes to raw coefficients.

Contents

Executive assessment **1**

1 Source snapshot and exclusion protocol **3**

2 A: one fixed modular certificate for all controller units **4**

3 B: make the LTE spectrum the primary interface **6**

4 C: characterize carries locally between established resets **8**

5 D: a quadratic substitution removes a Pell-bound induction **10**

6 Validation, trust, and source accounting **11**

7 A concrete adoption sequence **12**

A Archive guide and replay **13**

B Closest prior mechanisms and the retained difference **13**

1 Source snapshot and exclusion protocol

1.1 What was inspected

The source baseline is the immutable commit `9f35798db4a63c25e435fca14d75931144657d7e`, retrieved from `main` at the start of this review. The relevant project is `Computability/DiophantineConsistency`; the exclusion roots are its `research/lean-proof-simplification` directory and `docs/incoming`. The former contains the 23 placed packets a through w; the latter contains five current ZIP archives in addition to its intake README. The adoption ledger is useful evidence of what has reached the maintained tree, but adoption status does not determine exclusion here.[1, 2, 3, 4]

The audit records 1,292 Git member paths and blob identities, and 1,042 non-directory members of the five incoming archives. It verified 183 primary, index, and supporting text copies, totalling 1,924,159 bytes, against their pinned Git blobs. The five incoming byte streams were checked against their Git blob identities and also hashed with SHA-256. A consolidated ledger contains 155 exclusion entries; these overlap and are not a count of independent inventions.

The review inspected primary report structures, proposal catalogues, candidate-relevant arguments, and detailed close precedents, with focused searches of supporting notes, code, and patches. The complete member inventory must not be confused with a claim to have semantically read every historical snapshot, log, or binary PDF. Manuscript inspection used the LaTeX sources; archived PDFs were not separately rendered. The accompanying `audit/COVERAGE.md` and manifests state this scope explicitly.

A partial materialization encountered disk pressure in the late packet files. Exact primary copies for those packets were separately restored and verified; truncated working copies were not used as evidence. This is a provenance qualification, not an unresolved gap in the primary-manuscript comparison.

1.2 Meaning of "assume already applied"

All compatible earlier mathematical contracts are treated as available, even when the adoption ledger calls the corresponding patch pending, deferred, or uncompiled. No synthetic checkout combining every historical patch was built: some are alternative implementations of the same interface. The resulting question is whether each proposed mechanism adds something after the earlier interfaces are assumed present.

In particular, finite product-unit decomposition, general modular transport, scalar carry resets, explicit full-cycle carry formulae, uniqueness of cyclic normalization, and existing Pell refinements are excluded as new ideas. The controller proposal below uses that infrastructure, but its new content is the fixed residue of the *actual two-parameter Gamma family*. The locality proposal adds a reset-separated prefix characterization, not another global carry-existence or uniqueness argument.

An attractive seven-line residue proof for `CurrentWordOrigin2011.origin_reflected_zero` was rejected as a headline recommendation: the earlier physical-residue/support interface and quotient-homomorphism support separation already cover its mechanism. The archive records that decision rather than presenting a fresh target as a fresh idea.

2 A: one fixed modular certificate for all controller units

2.1 The existing proof obligation

The maintained source defines

$$d = 94517 = 47 \cdot 2011, \qquad p = 120199, \qquad \rho = 2^{47}, \qquad B = 2^{94517},$$

$$q_n = B^{p^n}, \qquad A_w = K + B^{p^w}, \qquad \Gamma(w,n) = \rho^{977}(q_n^2 - 1)(1 + q_n A_w).$$

The target is the coprimality of this natural-number coefficient with dp^n, for independent natural parameters w, n. The existing proof uses separate power reductions modulo 47, 2011, and 120199, separate residue calculations for factors, and several primewise coprimality results before assembling the final unit statement.[5]

This is a stronger candidate for structural simplification than merely shortening each of those three paths. The entire coefficient can be evaluated in one fixed quotient. Neither the growing power modulus nor the two exponent parameters need remain visible in the finite arithmetic certificate.

2.2 The algebraic mechanism

Lemma 2.1 (Iteration of a fixed power)**.** *Let x belong to a monoid, and let $a \in \mathbb{N}$. If $x^a = x$, then*

$$x^{a^n} = x \qquad (n \in \mathbb{N}).$$

Proof. For $n = 0$, the exponent is one. If the identity holds at n, then

$$x^{a^{n+1}} = x^{a^n a} = (x^{a^n})^a = x^a = x.$$

No commutativity, primality, or multiplicative-order theorem is used. □

The checked Lean proof is three lines after the theorem header:

```
induction n with
| zero => simp
| succ n ih => rw [pow_succ, pow_mul, ih, h]
```

The benefit comes from its concrete use in the quotient

$$M = dp = \boxed{11360848883}.$$

This modulus has the same prime support as dp^n for $n \geq 1$, and is still a sufficient unit certificate at $n = 0$. The lifting argument below does not require prime-support terminology or a primality hypothesis.

Proposition 2.2 (Fixed-modulus unit certificate)**.** *For natural numbers a, b, d, p, n, if $a \equiv b \pmod{dp}$ and $\gcd(b, dp) = 1$, then*

$$\gcd(a, dp^n) = 1.$$

Proof. Congruence transfers coprimality with dp from b to a. Hence a is coprime to d and to p. The latter implies coprimality with p^n; multiply the two coprimality statements. At $n = 0$, the power factor is one. Neither primality of p nor coprimality of d, p is needed. □

The checked implementation uses `ZMod.isUnit_iff_coprime`, `Nat.coprime_mul_iff_right`, and `Nat.Coprime.pow_right`.

2.3 The exact finite certificate

The verified residues are:

Quantity modulo $M = 11360848883$	Canonical residue
$B = 2^{94517}$	10287028452
$\rho = 2^{47}$	10653241607
K	1491904576
$\Gamma(w, n)$, all independent w, n	706142772

The kernel residue is not supplied as a new opaque assumption. The checked CRT proof reconstructs it from exactly the existing kernel-residue statements

$$K \equiv 26 \pmod{47}, \qquad K \equiv 1995 \pmod{2011}, \qquad K \equiv 114787 \pmod{120199}.$$

These are the types of `CurrentKernelResidues2011.K_cast_47`, `K_cast_2011`, and `K_cast_auxiliary`.[6] The numerical proof then establishes

$$10287028452^{120199} \equiv 10287028452 \pmod{M}.$$

The fixed-power lemma gives both $q_n \equiv B$ and $B^{p^w} \equiv B$ modulo M. Therefore

$$\begin{aligned}\Gamma(w, n) &\equiv \rho^{977}(B^2 - 1)(1 + B(K + B)) \\ &\equiv 706142772 \pmod{M}.\end{aligned}$$

Finally, the small integers satisfy the exact Bezout identity

$$(-567596732) \cdot 706142772 + 35279435 \cdot 11360848883 = 1.$$

Thus 706142772 is coprime to M, and the fixed-modulus unit certificate finishes the universal result.

The Lean proof uses `reduce_mod_char` in finite quotients rather than constructing the enormous natural power 2^{94517}. The independent Python script repeats the finite arithmetic with modular exponentiation and checks the Bezout identity. Its sample exponent pairs are useful controls; the universal claim comes from the Lean induction, not from sampling.

2.4 Natural subtraction and boundary cases

The expression $q_n^2 - 1$ is natural subtraction. Before transporting the expression to a ring, the proof establishes $1 \le q_n^2$ from $1 \le B$. Only then does it apply `Nat.cast_sub`. Dropping that step would silently change the polynomial interpretation when the base is too small.

The parameters remain independent. In particular, $(w, n) = (0, 0)$, $(1, 0)$, and $(0, 1)$ are all covered. The checked general lemma does not infer primality from a residue calculation. The current unit proof also remains separate from positivity of Γ, from its signed rank interpretation, and from the safe physical realization of rank increments.

2.5 Integration and expected benefit

`lean/GammaCertificate.lean` proves the actual numerical specialization from the three existing kernel-residue propositions. It does not import or use the old Gamma-unit conclusion or the old exact-order arguments. Its final result has the same natural expression in 2^{47}, 2^{94517}, K, and 120199 as the maintained coefficient.

`proposals/GammaRepositoryAdapter.lean` records the remaining definitional binding to `Gamma`, `q`, `A`, and `dimension`. This adapter is explicitly uncompiled; the full project dependency closure was not rebuilt. It imports the current module to obtain the existing definitions, so it is a reviewable binding sketch, not a claim that the current import graph has already been reduced.

The intended production organization is a small fixed-modulus certificate below the final controller-unit theorem, with old public residue lemmas retained as wrappers until their users are migrated. The main logical dependency then becomes

$$\text{existing literal } K \text{ residues} \implies \text{fixed Gamma residue} \implies \text{all power-modulus units.}$$

This is one coherent proposal. Counting the monoid lemma, the CRT step, and the coprimality lift as three independent new ideas would overstate the result.

No net source reduction is claimed for the supplied explanatory Gamma packet. The complete kernels deliberately expose all certificate steps. Removing old factor lemmas before checking their public clients would create an artificial line saving. The expected improvement is elimination of several different proof obligations and their dependencies from the principal unit argument.

3 B: make the LTE spectrum the primary interface

3.1 Reorient the proof, retaining the public results

The existing `PrimePowerOrder` module establishes the exact first lift, then exact lifting at prime-power exponents, then the multiplicative order. Its `order_succ` proof separates the smallest modulus from the successor case. The module is only 80 lines, so a claim of a huge global source reduction would be implausible.[7]

The proposed change makes the following arbitrary-exponent predicate the foundational lemma. Exact lifting and exact order become short consequences. This is the classical lifting-the-exponent mechanism already provided by Mathlib; the contribution is a better local proof interface and dependency direction.[8]

Proposition 3.1 (Complete divisibility spectrum)**.** *Suppose p is an odd prime and*

$$p \mid g - 1, \qquad p^2 \nmid g - 1.$$

Then for every $k, m \in \mathbb{N}$,

$$p^{k+1} \mid g^m - 1 \quad \iff \quad p^k \mid m. \tag{1}$$

Proof. The nondivisibility hypothesis forces $g > 1$: otherwise $g - 1 = 0$ in $\mathbb{N}$, which is divisible by every p^2. Also $p \nmid g$, because otherwise p would divide both g and $g - 1$.

Write v_p for extended multiplicity, taking $v_p(0) = \infty$. The two first-lift assumptions give $v_p(g-1) = 1$, and odd-prime LTE gives

$$v_p(g^m - 1) = 1 + v_p(m).$$

Consequently,

$$p^{k+1} \mid g^m - 1 \iff k + 1 \le 1 + v_p(m) \iff k \le v_p(m) \iff p^k \mid m.$$

The cancellation is of a finite one in $\mathbb{N} \cup \{\infty\}$, so it remains valid when $m = 0$. In that case both divisibility statements are true. □

The checked helper uses `Nat.emultiplicity_pow_sub_pow`, `pow_dvd_iff_le_emultiplicity`, and `ENat.add_le_add_iff_left`. It does not require a special branch for the zero exponent.

3.2 Exact order without a case split

The existing natural-cast bridge yields

$$(g : \mathrm{ZMod}(p^{k+1}))^m = 1 \quad \iff \quad p^k \mid m.$$

Since $\mathrm{orderOf}(x) \mid m$ is equivalent to $x^m = 1$, the numbers $\mathrm{orderOf}(g)$ and p^k divide exactly the same natural numbers. Evaluate the equivalence once at each number to obtain mutual divisibility and apply `Nat.dvd_antisymm`.

This proof works for $k = 0$: the order is one, and every exponent satisfies the power-equals-one condition. It also avoids constructing a separate nontrivial lower-power witness in the successor branch.

The old exact-lift theorem is recovered by substituting $m = p^k$ in (1), and then doing the same with modulus exponent $k + 2$. The second divisibility fails because $p^{k+1} \nmid p^k$. In the final candidate the whole old exact-lift body is one `simp` invocation using `Nat.pow_dvd_pow_iff_le_right`.

3.3 Why the interface matters

The previous module as a whole already implies (1): combine its exact order with its cast/divisibility bridge. Thus the proposal is *not* a stronger mathematical theory than the old module. It provides the general predicate directly from LTE and uses it to simplify the proof of order, avoiding that round trip.

Future clients asking whether a power equals one, whether a difference is divisible by a prime power, or whether two exponents agree modulo the order can start with the general predicate. Such downstream simplification is a plausible benefit, but no future client reduction is included in the measured counts.

3.4 Actual patch and validation

The production candidate preserves all six old theorem headers textually: `one_lt_of_not_square`, `not_dvd_of_dvd_sub_one`, `pow_cast_eq_one_iff`, `exact_lift`, `order_succ`, and `order_prime_power`. It adds `lift_dvd_iff`.

Measured region	Before	After
Whole production module, physical lines	80	75
`exact_lift` proof body	9	1
`order_succ` proof body	16	4

The five-line net saving charges the complete new helper, its statement, and comments. The larger proof-body reductions are not added again to that net saving.

The exact candidate compiled with Lean 4.32.0. The unaltered maintained `PrimePowerOrderChecks` examples were replayed against the candidate, and the six pre-existing theorem axiom sets match the baseline. The new helper uses only `propext`, `Classical.choice`, and `Quot.sound`. A separate standalone file checks $m = 0$, $k = 0$, and the exponents 18 and 6 modulo 27.

Oddness is essential. At $p = 2, g = 3, k = 2, m = 2$, one has $8 \mid 3^2 - 1$ but $4 \nmid 2$. The report and patch retain the odd-prime hypothesis rather than advertising an incorrect uniform statement.

4 C: characterize carries locally between established resets

4.1 The remaining opportunity after global carry proposals

The current general existence proof is already concise: it obtains a bounded carry assignment from a monotone fixed-point theorem and derives digit bounds and the transport congruence. Earlier proposals cover scalar reset lemmas, explicit extremal full-cycle carries, and normalization uniqueness. Replacing those by another generic fixed-point or uniqueness interface would repeat the excluded work.[9, 10]

The new opportunity is *local dependence after a reset*. For the actual assembled word, at whole-cell displacement, the maintained top-field proofs establish

$$\mathrm{raw}_j(2009) + 57 < \rho, \qquad \mathrm{raw}_j(2010) = 0, \qquad \rho = 2^{47}.$$

Every bounded normalization therefore has zero outgoing carry at fields 2009 and 2010.[11] This permits a finite-prefix description inside each raw cell. It lets proofs about unchanged raw prefixes avoid redoing a global normalization argument or expanding a rotated whole-word numerator.

4.2 A finite evaluator and its checked properties

For a natural array a, incoming seed s, and positive radix ρ, define

$$R_a(s, 0) = s, \qquad R_a(s, i + 1) = \left\lfloor \frac{a(i) + R_a(s, i)}{\rho} \right\rfloor.$$

Here $R_a(s,i)$ is the incoming carry to field i, or equivalently the outgoing carry of field $i-1$. This convention matters at the cell boundary.

Lemma 4.1 (Bounded finite prefix)**.** *If $s \le D$ and $a(i)+D < (D+1)\rho$ for $i < L$, then $R_a(s,i) \le D$ for every $i \le L$.*

Proof. Induct on i. The successor step uses $a(i) + R_a(s,i) \le a(i) + D < (D+1)\rho$, then the natural division inequality. □

Lemma 4.2 (Reset followed by a zero field)**.** *Under the preceding prefix budget, if $a(L) + D < \rho$ and $a(L+1) = 0$, then*

$$R_a(s, L+1) = 0, \qquad R_a(s, L+2) = 0.$$

Proof. The first quotient has numerator below ρ, so it is zero. The next numerator is zero plus zero. The compiled `terminal_zero` theorem exports the second equality; its proof explicitly establishes the first equality too. □

Lemma 4.3 (Characterization and prefix locality)**.** *Any sequence with the same initial carry and the same quotient recurrence on a finite prefix agrees with R_a there. If $a(i) = b(i)$ for every $i < L$, then $R_a(s,L) = R_b(s,L)$.*

Both claims follow by ordinary induction on the visited prefix and are checked in `matches_run` and `prefix_congr`. The current numeric instance `current_cell_terminal` uses $D = 57, L = 2009, \rho = 2^{47}$ and concludes that the carry after all 2011 fields is zero for every bounded incoming seed.

This is a synchronization statement. A general bounded block need not forget its incoming carry: for radix ten and constant raw digit nine, the seeds zero and one stay distinct under arbitrarily many updates. The prototype includes finite instances of that counterexample, so no unconditional forgetting is being assumed.

4.3 The proposed repository characterization

Keep the existing normalization (k, digits), and set

$$a_j(e) = \mathrm{raw}(c, \mathrm{displacement}(s))(\mathrm{base}(j) + e).$$

The desired adapter, for $0 \le e < 2011$, is

$$k(\mathrm{base}(j) + e) = R_{a_j}(0, e+1), \tag{2}$$

$$\mathit{digits}(\mathrm{base}(j) + e) = (a_j(e) + R_{a_j}(0,e)) \bmod \rho. \tag{3}$$

At $e = 0$, the incoming carry is zero because

$$\mathrm{base}(j) - 1 = \mathrm{base}(j-1) + 2010$$

in $\mathrm{ZMod}(N \cdot 2011)$, and the previous cell's final carry is zero by the existing top-field theorem. This identity must retain the wraparound cases $j = 0$ and $N = 1$. At positive e, the predecessor is in the same cell. The maintained recurrence and `matches_run` then give (2); the decomposition and remainder uniqueness give (3).

This adapter is a finite characterization of an independently supplied normalization. It is not proved by defining a new record whose fields simply assert the desired equations. Its proof uses the actual recurrence and the already established boundary reset.

4.4 Exactly what locality means

If two assembled inputs yield equal *raw* values through field e of cell j, then their normalized digit and outgoing carry at that field agree, using the same zero incoming boundary. The prefix needed for digit equality is $i < e + 1$, not merely $i < e$: the current raw value must also agree.

This does not imply independence of the original source cells. Convolution and whole-cell rotation can make a_j depend on other, even distant, source cells. A safe source-edit proof must first compute its influence on raw coefficients; only then may it use this locality theorem. The theorem gives no new mask acceptance, safe rank correction, or equation solution by itself.

The current reset inputs require Boolean dummy and marker maps, a nonzero cell count, and whole-cell displacement. They are not a proof of the same reset for arbitrary binary displacement W.

4.5 Why characterization should precede a new constructor

A fully explicit construction could normalize each raw cell with seed zero, use its two terminal resets to prove cyclic compatibility, and assemble `Facts`. The digit bound, decomposition, weak word bound, and transport would then reuse the existing generic results. Nevertheless, this is likely longer than the present single application of the fixed-point existence theorem.

The recommended first integration is therefore the characterization (2)–(3). It preserves the short existence proof and adds an interface for local comparison. It should be retained only if a concrete source-edit, mask, or controller client becomes shorter after paying for the adapter.

There is also a concrete import constraint: `CurrentWordTopBounds2011` imports `CurrentWordClause2011`, which depends on `CurrentCarryConstruction2011`. Importing the top-field module back into the carry-construction module would create a cycle. Put the adapter in a later module, or split raw reset geometry into an earlier acyclic dependency layer.

5 D: a quadratic substitution removes a Pell-bound induction

The maintained theorem `NativePellPower.sixteen_pow_bound` states

$$r \geq 2 \quad \Longrightarrow \quad 2^{2r+2} + 2 \leq 16^r.$$

Its proof uses induction on r, an exponent-shift identity, and a nonlinear arithmetic step. The useful observation is that both exponential expressions are powers of the same quantity.[12]

Set $t = 4^r$. Then $t \geq 16$, $2^{2r+2} = 4t$, and $16^r = t^2$. Thus the entire theorem reduces to

$$4t + 2 \leq t^2.$$

For example, $t(t-4) \geq 16 \cdot 12 > 2$ proves the inequality directly. The compiler-checked replacement body is:

```
have ht : 16 <= 4 ^ r := by
  simpa using (Nat.pow_le_pow_right (by decide : 0 < 4) hr)
have h : 4 ^ r * 4 + 2 <= 4 ^ r * 4 ^ r := by nlinarith
simpa [pow_add, pow_mul, <- mul_pow] using h
```

The displayed ASCII comparison/arrows are for typesetting; the actual patch contains the normal Lean symbols and the exact checked source.

This removes four physical proof-body lines, from eight to four, with no new production helper. It leaves the theorem statement and the $r \geq 2$ guard unchanged. The guard cannot simply be dropped: at $r = 1$, the proposed inequality would be $18 \leq 16$.

This is a deliberately small proposal. Earlier Pell work in the exclusion corpus concerns norm factoring, fixed-index anchors, and geometric-sum arguments. The present change is this specific reduction of a two-exponential comparison to a single quadratic inequality. It makes no claim about shortening all Pell proofs or improving elaboration time.

6 Validation, trust, and source accounting

6.1 What actually ran

The checked compiler is Lean 4.32.0, commit `8c9756b28d64dab099da31a4c09229a9e6a2ef35`, using the repository's pinned Mathlib revision. Existing compiler and dependency caches were reused without modifying their source trees. A missing Mathlib multiplicity object was built from source whose Git blob matches the pinned dependency. The archive contains source hashes, diagnostics, and portable replay scripts.

Artifact	Checked scope	Remaining scope
`FullLTESpectrum.lean`	General spectrum, cast bridge, order, exact lift, four focused examples	None within this standalone theorem set.
LTE production candidate	Exact module, old theorem headers, existing focused checks, old axiom-set comparison, patch applicability	Full project/client graph was not rebuilt.
`FixedPointCore.lean`	Monoid fixed power, coprimality transport/lift, polynomial specialization, two Pell presentations	The Pell production module and its clients were not rebuilt.
`GammaCertificate.lean`	CRT of the three K-residue assumptions, actual base/radix residues, all independent exponent pairs	Binding the literal project definitions remains an uncompiled adapter.
`ResetLocality.lean`	Bounded prefix, reset, recurrence characterization, prefix locality, current numeric terminal instance	The cyclic-coordinate adapter and its project clients are proposed.

The checked new source uses no `sorry`, no author-declared axiom, and no `native_decide`. Printed axiom sets are subsets of the usual `propext`, `Classical.choice`, and `Quot.sound`. The local carry

lemmas need only `propext` and `Quot.sound` in their reported sets. This is an ordinary Lean-kernel proof audit, not a claim that all statements are constructive or axiom-free.

The independent modular-arithmetic script checks finite constants and selected asymmetric exponent pairs. Those computations are useful for detecting a wrong modulus, swapped parameter, or copying error. They do not replace the universal Lean proof.

6.2 What the source counts mean

The LTE count compares the entire maintained file with the complete proposed replacement. It includes the new helper and retains all old theorem types. The Pell count compares the exact old and new proof bodies with the same theorem header. Their sum is nine removed production lines.

The accompanying source archive is longer because it includes explanatory kernels, independent examples, manifests, original source excerpts, and replay scripts. Those research artifacts are not proposed imports into the production module, and their size is not subtracted from a claimed production saving. Conversely, there is no numerical reduction claim for Gamma or locality until all necessary production helpers and retained compatibility results are counted.

No controlled runtime comparison was performed. A shorter proof can elaborate more slowly, and a finite numeric certificate can move cost into normalization. The reported outcomes are correctness and source-structure results only.

7 A concrete adoption sequence

1. **Apply the LTE patch to the pinned module.** Replay the unmodified focused checks and the axiom comparison. The patch keeps the six existing public signatures and adds the new predicate interface.
2. **Apply the Pell body replacement.** Rebuild `NativePellPower` and its direct clients in the normal repository workflow. The proof body already compiles in isolation with the unchanged type.
3. **Bind the Gamma certificate to the literal project constants.** Check the adapter, place the certificate below the unit theorem in the import graph, and verify that no old unit theorem is used in its proof. Retain or migrate public intermediate results before measuring final source size.
4. **Add one carry-characterization adapter after the top-field results.** Prove the cyclic seam once and derive the two local formulas. Use it in one concrete comparison client before considering a replacement constructor.

Each step is reviewable independently. None changes the literal Diophantine equation, the arithmetic constants, the meaning of the source system, or the status of the remaining equivalence proof. The report proposes proof simplifications; it does not close any of those separate research obligations.

A Archive guide and replay

The ZIP includes this compiled report and its LaTeX source, four checked Lean source files, exact LTE and Pell patches, the complete LTE candidate module, clearly labelled integration sketches, finite-certificate data, validation receipts, and the exclusion audit. Original target source files and immutable source links make the comparison reproducible.

The top-level `README.txt` gives the exact layout and commands. `scripts/verify_kernels.py` runs the standalone files using an existing checkout with the pinned Lean/Mathlib environment. It adds only a temporary object directory for the helper imported by the Gamma proof. `lte/verify_lte.py` separately compiles the exact LTE candidate and the maintained checks without changing the supplied checkout. `scripts/verify_gamma.py` reproduces the finite certificate with Python. The replay scripts are supplied as artifacts; the recorded validation receipts state which calls were actually run in this review.

The uncompiled adapters live under `proposals/`. They are deliberately not included in the standalone replay target, so an error in an integration sketch cannot be hidden by an otherwise successful kernel run.

B Closest prior mechanisms and the retained difference

Proposal	Earlier work assumed available	Additional content retained here
Gamma	Generic congruence interfaces, product-unit decomposition, literal residue data	The actual complete coefficient is constant modulo dp for independent w, n; a single fixed-point certificate removes the three power-reduction paths from its main proof.
LTE	Existing exact lift and exact order in the maintained module; no matched earlier simplification proposal	Reverse the proof dependency: arbitrary-exponent multiplicity predicate first, then the unchanged exact-lift and order results.
Carry locality	Scalar reset/decomposition (ledger E031), full-cycle extremal carry formulae (E066), uniqueness (E073), signed-prefix divmod and concatenation (E136/E154)	An existing normalization is determined by a bounded raw-cell prefix after a proved reset; compare local prefixes without a global cyclic formula. The signed-prefix precedent concerns borrow propagation; this adapter uses proved resets in the actual natural cyclic normalization.
Pell bound	Norm factoring, fixed-index-six anchor, geometric identities and geometric-sum induction cleanup	The exact `sixteen_pow_bound` statement becomes a quadratic inequality in 4^r, with a four-line checked body.

The audit also records rejected or low-priority candidates. In particular, origin-field residue simplification was not counted, and arbitrary fixed-cardinal subset translations were not included because the current interval API is already short and no net client saving was demonstrated.

References

[1] V. Reshetnikov, *ProveIt*, pinned commit `9f35798db4a63c25e435fca14d75931144657d7e`. https://github.com/VladimirReshetnikov/ProveIt/tree/9f35798db4a63c25e435fca14d75931144657d7e/Computability/DiophantineConsistency.

[2] *Prior Lean proof-simplification packets a–w*. https://github.com/VladimirReshetnikov/ProveIt/tree/9f35798db4a63c25e435fca14d75931144657d7e/Computability/DiophantineConsistency/research/lean-proof-simplification.

[3] *Incoming archives and intake README*. https://github.com/VladimirReshetnikov/ProveIt/tree/9f35798db4a63c25e435fca14d75931144657d7e/docs/incoming.

[4] *Lean proof-simplification adoption ledger*. https://github.com/VladimirReshetnikov/ProveIt/blob/9f35798db4a63c25e435fca14d75931144657d7e/Computability/DiophantineConsistency/research/lean-proof-simplification/ADOPTION.md.

[5] *CurrentControllerUnits2011.lean*, especially `power_dimension_47`, `power_dimension_2011`, `power_dimension_auxiliary`, and `coprime_Gamma_width_cells`. https://github.com/VladimirReshetnikov/ProveIt/blob/9f35798db4a63c25e435fca14d75931144657d7e/Computability/DiophantineConsistency/Lean/DiophantineConsistency/CurrentControllerUnits2011.lean.

[6] *CurrentKernelResidues2011.lean*. https://github.com/VladimirReshetnikov/ProveIt/blob/9f35798db4a63c25e435fca14d75931144657d7e/Computability/DiophantineConsistency/Lean/DiophantineConsistency/CurrentKernelResidues2011.lean.

[7] *PrimePowerOrder.lean* and its maintained checks. https://github.com/VladimirReshetnikov/ProveIt/blob/9f35798db4a63c25e435fca14d75931144657d7e/Computability/DiophantineConsistency/Lean/DiophantineConsistency/PrimePowerOrder.lean.

[8] Mathlib, *NumberTheory/Multiplicity.lean*, pinned revision `81a5d257c8e410db227a6665ed08f64fea08e997`. https://github.com/leanprover-community/mathlib4/blob/81a5d257c8e410db227a6665ed08f64fea08e997/Mathlib/NumberTheory/Multiplicity.lean.

[9] *NativeCyclicCarry.lean*. https://github.com/VladimirReshetnikov/ProveIt/blob/9f35798db4a63c25e435fca14d75931144657d7e/Computability/DiophantineConsistency/Lean/DiophantineConsistency/NativeCyclicCarry.lean.

[10] *CurrentCarryConstruction2011.lean*. https://github.com/VladimirReshetnikov/ProveIt/blob/9f35798db4a63c25e435fca14d75931144657d7e/Computability/DiophantineConsistency/Lean/DiophantineConsistency/CurrentCarryConstruction2011.lean.

[11] *CurrentWordTopBounds2011.lean*, especially `raw_2009_budget`, `raw_2010_zero`, and `top_carry_digits`. https://github.com/VladimirReshetnikov/ProveIt/blob/9f35798db4a63c25e435fca14d75931144657d7e/Computability/DiophantineConsistency/Lean/DiophantineConsistency/CurrentWordTopBounds2011.lean.

[12] *NativePellPower.lean*, `sixteen_pow_bound`. https://github.com/VladimirReshetnikov/ProveIt/blob/9f35798db4a63c25e435fca14d75931144657d7e/Computability/DiophantineConsistency/Lean/DiophantineConsistency/NativePellPower.lean.